Diseño e implementación de la base de datos nexa_servicios para la gestión del proceso de compras

Pedro Juan Campuzano Echeverri

Simón Atencio Posada

Corporación Universitaria Remington

Ingeniería de Software 2

Gloria Amparo Lora

1 de noviembre de 2026

Diseño e implementación de la base de datos nexa_servicios para la gestión del proceso de compras

Este documento describe cómo se construyó la base de datos `nexa_servicios` y cómo funciona. La base de datos fue creada en SQL sobre el gestor MariaDB, trabajando directamente desde la consola de comandos (cliente `mysql` de MariaDB). Su finalidad es almacenar y controlar el ciclo completo de una compra dentro de una organización: desde que un usuario registra una solicitud hasta que se recibe lo comprado y se verifica la factura del proveedor.

Se utilizó el motor de almacenamiento InnoDB en todas las tablas, ya que permite definir llaves foráneas y mantener la integridad referencial entre los registros. Además, la base de datos se creó con el conjunto de caracteres `utf8mb4`, que admite tildes, eñes y otros símbolos sin problemas de codificación.

Descripción general de la base de datos

La base de datos quedó conformada por 22 tablas, que se pueden agrupar en cuatro tipos según la función que cumplen: tablas de catálogo, tablas maestras, tablas del proceso de compra y una tabla de auditoría. La Tabla 1 presenta el resumen de cada una con su número de columnas.

Tabla 1

Resumen de las tablas de la base de datos nexa_servicios

Grupo	Tabla	Función	Columnas
Catálogo	roles	Perfiles de acceso de los usuarios	4
Catálogo	estados_solicitud	Estados de una solicitud de compra	3
Catálogo	prioridades	Niveles de prioridad de una solicitud	3
Catálogo	tipos_item	Producto o servicio	3
Catálogo	tipos_decision	Decisiones de una aprobación	3
Catálogo	estados_cotizacion	Estados de una cotización	3
Catálogo	estados_orden	Estados de una orden de compra	3

Grupo	Tabla	Función	Columnas
Catálogo	estados_recepcion	Estados de una recepción	3
Catálogo	estados_factura	Estados de una factura	3
Maestra	usuarios	Personas que usan el sistema	10
Maestra	proveedores	Empresas que venden o cotizan	9
Proceso	solicitudes_compra	Encabezado de la solicitud	10
Proceso	detalle_solicitud	Ítems solicitados	7
Proceso	aprobaciones	Decisiones sobre las solicitudes	6
Proceso	cotizaciones	Ofertas de los proveedores	14
Proceso	detalle_cotizacion	Precios por ítem cotizado	6
Proceso	ordenes_compra	Órdenes generadas al proveedor	12
Proceso	detalle_orden_compra	Ítems de cada orden	8
Proceso	recepciones	Entrega de lo comprado	7
Proceso	detalle_recepcion	Cantidades recibidas y aceptadas	7
Proceso	facturas	Facturas del proveedor	11
Auditoría	auditoria	Bitácora de acciones sobre los datos	7

Nota. Elaboración propia a partir de la estructura final obtenida con el comando DESCRIBE.

Proceso de construcción

Entorno de trabajo y creación de la base de datos

Primero se ingresó al cliente de MariaDB y se ejecutó `SHOW DATABASES;` para ver qué bases de datos existían en el servidor (por ejemplo `tienda_porcicarnes` o `sistema_tutorial`) y asegurar que el nombre elegido estuviera libre. Luego se creó la base de datos y se seleccionó para trabajar sobre ella:

```
CREATE DATABASE nexa_servicios
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE nexa_servicios;
SELECT DATABASE();
```

En un primer intento se usó la intercalación `utf8mb4_general_ci`, pero en la versión final se cambió a `utf8mb4_unicode_ci` porque ordena y compara los textos en español de

forma más precisa. Cada vez que fue necesario empezar de cero se utilizó `DROP DATABASE IF EXISTS nexa_servicios;`, que elimina la base solo si existe y evita errores.

Construcción por etapas y corrección de errores

La base de datos no se escribió de una sola vez. Se construyó tabla por tabla, comprobando cada resultado con `SHOW TABLES;` y `DESCRIBE nombre_tabla;`. Durante el proceso aparecieron varios errores, que ayudaron a entender mejor la sintaxis y el orden correcto de creación. La Tabla 2 recoge los más importantes.

Tabla 2

Errores presentados durante la construcción y su solución

Error	Causa	Solución aplicada
1064 (sintaxis)	Sentencia `CREATE TABLE usuarios` cerrada antes de tiempo con `;` y un punto `.` en lugar de coma después de `telefono VARCHAR(20).`	Se reescribió la sentencia completa con las comas y el cierre `) ENGINE=InnoDB;.`
1064 (sintaxis)	La palabra reservada se escribió `FOREING KEY` en lugar de `FOREIGN KEY`, y quedó una columna suelta (`rec`).	Se corrigió la ortografía y se eliminó el texto sobrante.
1064 (sintaxis)	Faltaba una letra en `PRIMARY KEY` (se escribió `RIMARY KEY`) y se usaron comandos incompletos como `DROP TABLES;`, `DROP nexa_servicios;` y `SHOW TBLES;.`	Se usaron los comandos completos: `PRIMARY KEY`, `DROP DATABASE IF EXISTS` y `SHOW TABLES`.
1005, errno 150	La llave foránea quedó mal formada porque la tabla `usuarios` no se había creado (falló por el error anterior). Como consecuencia fallaron en cadena todas las tablas que dependían de ella.	Se eliminó la base de datos y se volvió a crear respetando el orden de dependencias (Tabla 4).
1146	Se intentó insertar datos en `roles` cuando la tabla no existía.	Se crearon primero las tablas y después se insertaron los datos.
1050	Se ejecutó `CREATE TABLE roles` cuando la tabla ya existía.	Se verificó con `SHOW TABLES;` antes de crear.
1062	Se ejecutó dos veces el `INSERT` de `estados_orden`, repitiendo valores que son `UNIQUE`.	No fue necesaria otra acción: la restricción `UNIQUE` evitó los duplicados, lo que confirma que funciona.

Nota. Elaboración propia a partir del historial de la consola de MariaDB.

Decisiones de rediseño

Entre la primera versión y la versión final se hicieron varios cambios para que el modelo fuera más ordenado y evitara errores de digitación. La Tabla 3 los resume.

Tabla 3

Cambios entre la primera versión y la versión final del diseño

Aspecto	Primera versión	Versión final
Estados de cotización, orden, recepción y factura	Columna `estado` de texto libre (`VARCHAR`).	Tablas de catálogo (`estados_*`) enlazadas con llave foránea.
Decisión de una aprobación	Texto validado con `CHECK (decision IN (...))`.	Tabla `tipos_decision` enlazada con llave foránea.
Contraseña del usuario	Columna `contrasena`.	Columna `contrasena_hash`, pensada para guardar la contraseña cifrada.
Totales de cotización, orden y factura	`valor_total` digitado manualmente.	Columna generada: subtotal − descuento + impuestos.
Subtotales de los detalles	Digitados manualmente.	Columnas generadas (cantidad × precio unitario).
Cantidad rechazada en la recepción	Digitada manualmente.	Columna generada: recibida − aceptada.
Reglas de las llaves foráneas	`ON UPDATE CASCADE`, `ON DELETE RESTRICT`, `CASCADE` o `SET NULL`.	Comportamiento por defecto (`RESTRICT`), que impide borrar datos con historial.
Número de factura	`UNIQUE` sobre `numero_factura`.	`UNIQUE` sobre la pareja `(id_proveedor, numero_factura)`.
Tabla `auditoria`	`id_usuario` obligatorio e `id_registro` de tipo `INT`.	`id_usuario` opcional, `id_registro` de tipo `BIGINT` e índices de búsqueda.
Detalle de la orden de compra	Solo se enlazaba con el detalle de la solicitud.	Se agregó `id_detalle_cotizacion` para saber de qué cotización viene cada precio.
Tabla `ordenes_compra`	Incluía `id_proveedor`.	Se quitó: el proveedor se obtiene a través de la cotización.
Tabla `cotizaciones`	Solo `valor_total` y `estado`.	Se agregaron vencimiento, tiempo de entrega, condiciones de pago, descuento e impuestos.

Nota. Elaboración propia.

Orden de creación de las tablas

Como una llave foránea solo puede apuntar a una tabla que ya existe, las tablas se crearon por niveles de dependencia: primero las que no dependen de ninguna otra y al final las que dependen de varias. Este orden fue la solución al error 1005 de la Tabla 2 y se muestra en la Tabla 4.

Tabla 4

Orden de creación de las tablas según sus dependencias

Nivel	Tablas creadas	Depende de
1	roles, proveedores, estados_solicitud, prioridades, tipos_item, tipos_decision, estados_cotizacion, estados_orden, estados_recepcion, estados_factura	Ninguna
2	usuarios	roles
3	solicitudes_compra	usuarios, estados_solicitud, prioridades
4	detalle_solicitud, aprobaciones, cotizaciones	solicitudes_compra y catálogos
5	detalle_cotizacion, ordenes_compra	cotizaciones, detalle_solicitud, usuarios
6	detalle_orden_compra, recepciones, facturas	ordenes_compra
7	detalle_recepcion	recepciones, detalle_orden_compra
8	auditoria	usuarios

Nota. Elaboración propia.

Script de la base de datos

A continuación se presenta el script final tal como quedó ejecutado, organizado por grupos de tablas. En todas las tablas se usó `ENGINE=InnoDB` y las llaves primarias son números enteros con `AUTO_INCREMENT`, de modo que el motor asigna el identificador automáticamente.

Tablas de catálogo

Los catálogos guardan listas de valores que se repiten en el sistema (roles, estados, prioridades). Todos tienen la misma estructura: un identificador, un nombre único y una

descripción opcional. La restricción `UNIQUE` en el nombre impide registrar dos veces el mismo valor.

```
CREATE TABLE roles (
    id_rol INT AUTO_INCREMENT PRIMARY KEY,
    nombre_rol VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;

CREATE TABLE estados_solicitud (
    id_estado_solicitud INT AUTO_INCREMENT PRIMARY KEY,
    nombre_estado VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE prioridades (
    id_prioridad INT AUTO_INCREMENT PRIMARY KEY,
    nombre_prioridad VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE tipos_item (
    id_tipo_item INT AUTO_INCREMENT PRIMARY KEY,
    nombre_tipo VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE tipos_decision (
    id_tipo_decision INT AUTO_INCREMENT PRIMARY KEY,
    nombre_decision VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE estados_cotizacion (
    id_estado_cotizacion INT AUTO_INCREMENT PRIMARY KEY,
    nombre_estado VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE estados_orden (
    id_estado_orden INT AUTO_INCREMENT PRIMARY KEY,
    nombre_estado VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE estados_recepcion (
    id_estado_recepcion INT AUTO_INCREMENT PRIMARY KEY,
    nombre_estado VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;

CREATE TABLE estados_factura (
    id_estado_factura INT AUTO_INCREMENT PRIMARY KEY,
    nombre_estado VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NULL
) ENGINE=InnoDB;
```

Tablas maestras

Usuarios

La tabla `usuarios` almacena a las personas que usan el sistema. El documento y el correo son únicos, es decir, no pueden repetirse. La columna `id_rol` es una llave foránea hacia `roles`, que define qué puede hacer cada usuario. La fecha de registro se llena sola con la fecha y hora actuales.

```
CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    id_rol INT NOT NULL,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    documento VARCHAR(20) NOT NULL UNIQUE,
    correo VARCHAR(150) NOT NULL UNIQUE,
    telefono VARCHAR(20) NULL,
    contrasena_hash VARCHAR(255) NOT NULL,
    estado BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_usuarios_roles
        FOREIGN KEY (id_rol) REFERENCES roles(id_rol)
) ENGINE=InnoDB;
```

Proveedores

La tabla `proveedores` guarda las empresas a las que se les pide cotización o se les compra. Cada proveedor tiene un código y una identificación fiscal únicos, para no registrarlo dos veces.

```
CREATE TABLE proveedores (
    id_proveedor INT AUTO_INCREMENT PRIMARY KEY,
    codigo_proveedor VARCHAR(20) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL,
    identificacion_fiscal VARCHAR(30) NOT NULL UNIQUE,
    telefono VARCHAR(20) NOT NULL,
    correo VARCHAR(150) NULL,
    direccion VARCHAR(200) NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tablas del proceso de compra

Solicitud de compra y su detalle

`solicitudes_compra` es el punto de partida del proceso. Registra quién solicita (`id_solicitante`), quién la atiende (`id_responsable`, que es opcional), su estado, su prioridad, la descripción y la justificación. La tabla `detalle_solicitud` guarda cada ítem pedido; una solicitud puede tener varios ítems. Las restricciones `CHECK` obligan a que la cantidad sea mayor que cero y el valor estimado, si existe, no sea negativo.

```
CREATE TABLE solicitudes_compra (
    id_solicitud INT AUTO_INCREMENT PRIMARY KEY,
    codigo_solicitud VARCHAR(20) NOT NULL UNIQUE,
    id_solicitante INT NOT NULL,
    id_responsable INT NULL,
    id_estado_solicitud INT NOT NULL,
    id_prioridad INT NOT NULL,
    descripcion TEXT NOT NULL,
    justificacion TEXT NOT NULL,
    fecha_solicitud DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion DATETIME NULL,
    CONSTRAINT fk_solicitudes_solicitante
        FOREIGN KEY (id_solicitante) REFERENCES usuarios(id_usuario),
    CONSTRAINT fk_solicitudes_responsable
        FOREIGN KEY (id_responsable) REFERENCES usuarios(id_usuario),
    CONSTRAINT fk_solicitudes_estado
        FOREIGN KEY (id_estado_solicitud)
        REFERENCES estados_solicitud(id_estado_solicitud),
    CONSTRAINT fk_solicitudes_prioridad
        FOREIGN KEY (id_prioridad) REFERENCES prioridades(id_prioridad)
) ENGINE=InnoDB;

CREATE TABLE detalle_solicitud (
    id_detalle_solicitud INT AUTO_INCREMENT PRIMARY KEY,
    id_solicitud INT NOT NULL,
    id_tipo_item INT NOT NULL,
    descripcion_item VARCHAR(255) NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    unidad_medida VARCHAR(30) NULL,
    valor_estimado DECIMAL(15,2) NULL,
    CONSTRAINT chk_detalle_solicitud_cantidad CHECK (cantidad > 0),
    CONSTRAINT chk_detalle_solicitud_valor
        CHECK (valor_estimado IS NULL OR valor_estimado >= 0),
    CONSTRAINT fk_detalle_solicitud_solicitud
        FOREIGN KEY (id_solicitud) REFERENCES solicitudes_compra(id_solicitud),
    CONSTRAINT fk_detalle_solicitud_tipo
        FOREIGN KEY (id_tipo_item) REFERENCES tipos_item(id_tipo_item)
) ENGINE=InnoDB;
```

Aprobaciones

Cada vez que un aprobador toma una decisión sobre una solicitud (aprobar, rechazar o devolver) se guarda un registro en `aprobaciones`, con el usuario que decidió, el tipo de decisión, el motivo y la fecha. Así queda el historial completo de decisiones de cada solicitud.

```
CREATE TABLE aprobaciones (
    id_aprobacion INT AUTO_INCREMENT PRIMARY KEY,
    id_solicitud INT NOT NULL,
    id_usuario INT NOT NULL,
    id_tipo_decision INT NOT NULL,
    motivo TEXT NULL,
    fecha_decision DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_aprobaciones_solicitud
        FOREIGN KEY (id_solicitud) REFERENCES solicitudes_compra(id_solicitud),
    CONSTRAINT fk_aprobaciones_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    CONSTRAINT fk_aprobaciones_decision
        FOREIGN KEY (id_tipo_decision)
        REFERENCES tipos_decision(id_tipo_decision)
) ENGINE=InnoDB;
```

Cotizaciones y su detalle

Una vez aprobada la solicitud, el comprador pide cotizaciones a los proveedores. En `cotizaciones` se guardan las condiciones de la oferta: fechas, tiempo de entrega, forma de pago, subtotal, descuento e impuestos. La columna `valor_total` es una columna generada (`STORED`): MariaDB la calcula sola como subtotal menos descuento más impuestos y la almacena, por lo que nadie tiene que digitarla. En `detalle_cotizacion` se registra el precio unitario de cada ítem de la solicitud; la restricción `UNIQUE` evita cotizar dos veces el mismo ítem dentro de la misma cotización y el subtotal también se calcula automáticamente.

```
CREATE TABLE cotizaciones (
    id_cotizacion INT AUTO_INCREMENT PRIMARY KEY,
    codigo_cotizacion VARCHAR(30) NOT NULL UNIQUE,
    id_solicitud INT NOT NULL,
    id_proveedor INT NOT NULL,
    fecha_cotizacion DATE NOT NULL DEFAULT (CURRENT_DATE),
    fecha_vencimiento DATE NULL,
    tiempo_entrega_dias INT NULL,
    condiciones_pago VARCHAR(100) NULL,
    valor_subtotal DECIMAL(15,2) NOT NULL DEFAULT 0,
    descuento DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_impuestos DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_total DECIMAL(15,2)
```

```
        AS (valor_subtotal - descuento + valor_impuestos) STORED,
    id_estado_cotizacion INT NOT NULL,
    observaciones TEXT NULL,
    CONSTRAINT chk_cotizacion_vencimiento
        CHECK (fecha_vencimiento IS NULL
               OR fecha_vencimiento >= fecha_cotizacion),
    CONSTRAINT chk_cotizacion_entrega
        CHECK (tiempo_entrega_dias IS NULL OR tiempo_entrega_dias >= 0),
    CONSTRAINT chk_cotizacion_subtotal CHECK (valor_subtotal >= 0),
    CONSTRAINT chk_cotizacion_descuento CHECK (descuento >= 0),
    CONSTRAINT chk_cotizacion_impuestos CHECK (valor_impuestos >= 0),
    CONSTRAINT fk_cotizaciones_solicitud
        FOREIGN KEY (id_solicitud) REFERENCES solicitudes_compra(id_solicitud),
    CONSTRAINT fk_cotizaciones_proveedor
        FOREIGN KEY (id_proveedor) REFERENCES proveedores(id_proveedor),
    CONSTRAINT fk_cotizaciones_estado
        FOREIGN KEY (id_estado_cotizacion)
        REFERENCES estados_cotizacion(id_estado_cotizacion)
) ENGINE=InnoDB;

CREATE TABLE detalle_cotizacion (
    id_detalle_cotizacion INT AUTO_INCREMENT PRIMARY KEY,
    id_cotizacion INT NOT NULL,
    id_detalle_solicitud INT NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    precio_unitario DECIMAL(15,2) NOT NULL,
    subtotal DECIMAL(15,2) AS (cantidad * precio_unitario) STORED,
    CONSTRAINT uq_cotizacion_detalle
        UNIQUE (id_cotizacion, id_detalle_solicitud),
    CONSTRAINT chk_det_cot_cantidad CHECK (cantidad > 0),
    CONSTRAINT chk_det_cot_precio CHECK (precio_unitario >= 0),
    CONSTRAINT fk_det_cot_cotizacion
        FOREIGN KEY (id_cotizacion) REFERENCES cotizaciones(id_cotizacion),
    CONSTRAINT fk_det_cot_solicitud
        FOREIGN KEY (id_detalle_solicitud)
        REFERENCES detalle_solicitud(id_detalle_solicitud)
) ENGINE=InnoDB;
```

Órdenes de compra y su detalle

Cuando se elige una cotización, se genera la orden de compra. `ordenes_compra` se relaciona con la solicitud, con la cotización elegida y con el usuario comprador; el proveedor no se repite aquí porque ya está en la cotización. En `detalle_orden_compra` cada línea guarda de dónde viene: el ítem de la solicitud y el ítem de la cotización. Los totales y subtotales también son columnas generadas.

```
CREATE TABLE ordenes_compra (
    id_orden INT AUTO_INCREMENT PRIMARY KEY,
    codigo_orden VARCHAR(30) NOT NULL UNIQUE,
    id_solicitud INT NOT NULL,
    id_cotizacion INT NOT NULL,
    id_comprador INT NOT NULL,
    fecha_orden DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    valor_subtotal DECIMAL(15,2) NOT NULL DEFAULT 0,
    descuento DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_impuestos DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_total DECIMAL(15,2)
        AS (valor_subtotal - descuento + valor_impuestos) STORED,
    id_estado_orden INT NOT NULL,
    observaciones TEXT NULL,
    CONSTRAINT chk_orden_subtotal CHECK (valor_subtotal >= 0),
    CONSTRAINT chk_orden_descuento CHECK (descuento >= 0),
    CONSTRAINT chk_orden_impuestos CHECK (valor_impuestos >= 0),
    CONSTRAINT fk_orden_solicitud
        FOREIGN KEY (id_solicitud) REFERENCES solicitudes_compra(id_solicitud),
    CONSTRAINT fk_orden_cotizacion
        FOREIGN KEY (id_cotizacion) REFERENCES cotizaciones(id_cotizacion),
    CONSTRAINT fk_orden_comprador
        FOREIGN KEY (id_comprador) REFERENCES usuarios(id_usuario),
    CONSTRAINT fk_orden_estado
        FOREIGN KEY (id_estado_orden) REFERENCES estados_orden(id_estado_orden)
) ENGINE=InnoDB;

CREATE TABLE detalle_orden_compra (
    id_detalle_orden INT AUTO_INCREMENT PRIMARY KEY,
    id_orden INT NOT NULL,
    id_detalle_solicitud INT NOT NULL,
    id_detalle_cotizacion INT NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    precio_unitario DECIMAL(15,2) NOT NULL,
    descuento DECIMAL(15,2) NOT NULL DEFAULT 0,
    subtotal DECIMAL(15,2)
        AS (cantidad * precio_unitario - descuento) STORED,
    CONSTRAINT uq_orden_detalle_solicitud
        UNIQUE (id_orden, id_detalle_solicitud),
    CONSTRAINT chk_det_orden_cantidad CHECK (cantidad > 0),
    CONSTRAINT chk_det_orden_precio CHECK (precio_unitario >= 0),
    CONSTRAINT chk_det_orden_descuento CHECK (descuento >= 0),
    CONSTRAINT fk_det_orden_orden
        FOREIGN KEY (id_orden) REFERENCES ordenes_compra(id_orden),
    CONSTRAINT fk_det_orden_solicitud
        FOREIGN KEY (id_detalle_solicitud)
        REFERENCES detalle_solicitud(id_detalle_solicitud),
    CONSTRAINT fk_det_orden_cotizacion
        FOREIGN KEY (id_detalle_cotizacion)
        REFERENCES detalle_cotizacion(id_detalle_cotizacion)
) ENGINE=InnoDB;
```

Recepciones y su detalle

Cuando llega lo comprado, el receptor lo registra en `recepciones`, indicando si está conforme (`conforme`, que por defecto es falso hasta que se confirme). En `detalle_recepcion` se anota, por cada ítem de la orden, la cantidad recibida y la aceptada. La cantidad rechazada la calcula MariaDB (recibida menos aceptada) y una restricción `CHECK` impide que se acepte más de lo que se recibió.

```
CREATE TABLE recepciones (
    id_recepcion INT AUTO_INCREMENT PRIMARY KEY,
    id_orden INT NOT NULL,
    id_usuario INT NOT NULL,
    fecha_recepcion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_estado_recepcion INT NOT NULL,
    conforme BOOLEAN NOT NULL DEFAULT FALSE,
    observaciones TEXT NULL,
    CONSTRAINT fk_recepciones_orden
        FOREIGN KEY (id_orden) REFERENCES ordenes_compra(id_orden),
    CONSTRAINT fk_recepciones_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    CONSTRAINT fk_recepciones_estado
        FOREIGN KEY (id_estado_recepcion)
        REFERENCES estados_recepcion(id_estado_recepcion)
) ENGINE=InnoDB;

CREATE TABLE detalle_recepcion (
    id_detalle_recepcion INT AUTO_INCREMENT PRIMARY KEY,
    id_recepcion INT NOT NULL,
    id_detalle_orden INT NOT NULL,
    cantidad_recibida DECIMAL(10,2) NOT NULL,
    cantidad_aceptada DECIMAL(10,2) NOT NULL,
    cantidad_rechazada DECIMAL(10,2)
        AS (cantidad_recibida - cantidad_aceptada) STORED,
    observacion TEXT NULL,
    CONSTRAINT chk_det_rec_recibida CHECK (cantidad_recibida >= 0),
    CONSTRAINT chk_det_rec_aceptada
        CHECK (cantidad_aceptada >= 0
               AND cantidad_aceptada <= cantidad_recibida),
    CONSTRAINT fk_det_rec_recepcion
        FOREIGN KEY (id_recepcion) REFERENCES recepciones(id_recepcion),
    CONSTRAINT fk_det_rec_orden
        FOREIGN KEY (id_detalle_orden)
        REFERENCES detalle_orden_compra(id_detalle_orden)
) ENGINE=InnoDB;
```

Facturas

La tabla `facturas` registra la factura que envía el proveedor por una orden de compra. Un mismo número de factura puede repetirse entre proveedores distintos, pero no dentro del mismo proveedor; por eso la restricción `UNIQUE` se definió sobre la pareja proveedor y número. El total también se calcula automáticamente.

```
CREATE TABLE facturas (
    id_factura INT AUTO_INCREMENT PRIMARY KEY,
    numero_factura VARCHAR(50) NOT NULL,
    id_proveedor INT NOT NULL,
    id_orden INT NOT NULL,
    fecha_factura DATE NOT NULL DEFAULT (CURRENT_DATE),
    valor_subtotal DECIMAL(15,2) NOT NULL,
    descuento DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_impuestos DECIMAL(15,2) NOT NULL DEFAULT 0,
    valor_total DECIMAL(15,2)
        AS (valor_subtotal - descuento + valor_impuestos) STORED,
```

```
    id_estado_factura INT NOT NULL,
    observaciones TEXT NULL,
    CONSTRAINT uq_factura_proveedor_numero
        UNIQUE (id_proveedor, numero_factura),
    CONSTRAINT chk_factura_subtotal CHECK (valor_subtotal >= 0),
    CONSTRAINT chk_factura_descuento CHECK (descuento >= 0),
    CONSTRAINT chk_factura_impuestos CHECK (valor_impuestos >= 0),
    CONSTRAINT fk_facturas_proveedor
        FOREIGN KEY (id_proveedor) REFERENCES proveedores(id_proveedor),
    CONSTRAINT fk_facturas_orden
        FOREIGN KEY (id_orden) REFERENCES ordenes_compra(id_orden),
    CONSTRAINT fk_facturas_estado
        FOREIGN KEY (id_estado_factura)
        REFERENCES estados_factura(id_estado_factura)
) ENGINE=InnoDB;
```

Tabla de auditoría

La tabla `auditoria` funciona como bitácora: guarda qué usuario hizo una acción, sobre qué tabla y registro, de qué tipo fue (`INSERT`, `UPDATE` o `DELETE`) y cuándo ocurrió. El campo `id_usuario` es opcional para poder registrar también acciones del sistema. Se crearon índices sobre la tabla, el registro, la acción y la fecha para que las búsquedas en la bitácora sean rápidas.

```
CREATE TABLE auditoria (
    id_auditoria BIGINT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NULL,
    tabla_afectada VARCHAR(100) NOT NULL,
    id_registro BIGINT NOT NULL,
    accion VARCHAR(20) NOT NULL,
    fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    descripcion TEXT NULL,
    CONSTRAINT chk_auditoria_accion
        CHECK (accion IN ('INSERT','UPDATE','DELETE')),
    CONSTRAINT fk_auditoria_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    INDEX idx_auditoria_tabla (tabla_afectada),
    INDEX idx_auditoria_registro (id_registro),
    INDEX idx_auditoria_accion (accion),
    INDEX idx_auditoria_fecha (fecha_hora)
) ENGINE=InnoDB;
```

Carga de datos iniciales

Después de crear las tablas se cargaron los valores base de los catálogos con sentencias `INSERT`, y se comprobaron con `SELECT * FROM` cada tabla. Los valores insertados se muestran en la Tabla 5.

```
INSERT INTO estados_solicitud (nombre_estado) VALUES
('Pendiente'), ('En revision'), ('Aprobada'), ('Rechazada'),
('Devuelta'), ('En cotizacion'), ('Comprada'), ('Cerrada');
```

```
INSERT INTO prioridades (nombre_prioridad) VALUES
('Baja'), ('Media'), ('Alta'), ('Urgente');

INSERT INTO tipos_item (nombre_tipo) VALUES
('Producto'), ('Servicio');

INSERT INTO tipos_decision (nombre_decision) VALUES
('Aprobada'), ('Rechazada'), ('Devuelta');

INSERT INTO estados_cotizacion (nombre_estado) VALUES
('Recibida'), ('Seleccionada'), ('Descartada');

INSERT INTO estados_orden (nombre_estado) VALUES
('Generada'), ('Enviada'), ('Recibida'), ('Cerrada'), ('Cancelada');

INSERT INTO estados_recepcion (nombre_estado) VALUES
('Pendiente'), ('Recibida'), ('Conforme'), ('Con inconsistencias');

INSERT INTO estados_factura (nombre_estado) VALUES
('Pendiente'), ('Verificada'), ('Rechazada');
```

Tabla 5

Datos iniciales cargados en las tablas de catálogo

Tabla	Registros	Cantidad
estados_solicitud	Pendiente, En revision, Aprobada, Rechazada, Devuelta, En cotizacion, Comprada, Cerrada	8
prioridades	Baja, Media, Alta, Urgente	4
tipos_item	Producto, Servicio	2
tipos_decision	Aprobada, Rechazada, Devuelta	3
estados_cotizacion	Recibida, Seleccionada, Descartada	3
estados_orden	Generada, Enviada, Recibida, Cerrada, Cancelada	5
estados_recepcion	Pendiente, Recibida, Conforme, Con inconsistencias	4
estados_factura	Pendiente, Verificada, Rechazada	3

Nota. Elaboración propia a partir de los resultados de la consola.

Para la tabla `roles` se definieron los siguientes perfiles de acceso, que corresponden a las personas que intervienen en el proceso de compra:

```
INSERT INTO roles (nombre_rol, descripcion, activo) VALUES
('Administrador', 'Gestiona usuarios y configuracion del sistema', TRUE),
('Solicitante', 'Registra solicitudes de compra', TRUE),
('Comprador', 'Gestiona proveedores, cotizaciones y ordenes', TRUE),
('Aprobador', 'Aprueba o rechaza solicitudes', TRUE),
('Receptor', 'Registra la recepcion de compras', TRUE);
```

Funcionamiento de la base de datos

Flujo del proceso de compra

La base de datos sigue el recorrido real de una compra. Cada etapa deja un registro en una tabla y esa tabla se enlaza con la etapa anterior mediante llaves foráneas, de manera que siempre se puede rastrear una factura hasta la solicitud que la originó. La Tabla 6 resume el flujo.

Tabla 6

Etapas del proceso de compra y tablas que intervienen

Etapa	Rol que actúa	Tablas que se llenan	Resultado
1. Registro de usuarios	Administrador	roles, usuarios	Personas habilitadas con un rol
2. Solicitud	Solicitante	solicitudes_compra, detalle_solicitud	Solicitud en estado Pendiente con sus ítems
3. Revisión y aprobación	Aprobador	aprobaciones	Decisión: Aprobada, Rechazada o Devuelta
4. Cotización	Comprador	proveedores, cotizaciones, detalle_cotizacion	Ofertas comparables de varios proveedores
5. Orden de compra	Comprador	ordenes_compra, detalle_orden_compra	Orden basada en la cotización seleccionada
6. Recepción	Receptor	recepciones, detalle_recepcion	Cantidades recibidas, aceptadas y rechazadas
7. Facturación	Comprador	facturas	Factura asociada a la orden y al proveedor
8. Control	Sistema	auditoria	Registro de quién hizo qué y cuándo

Nota. Elaboración propia.

El avance de la solicitud se refleja en la columna `id_estado_solicitud`, que recorre los valores del catálogo: Pendiente, En revisión, Aprobada (o Rechazada o Devuelta), En cotización, Comprada y Cerrada. Lo mismo ocurre con las cotizaciones (Recibida, Seleccionada, Descartada), las órdenes (Generada, Enviada, Recibida, Cerrada, Cancelada), las recepciones y las facturas. El script no incluye disparadores ni procedimientos almacenados, por lo que el

cambio de estado se hace con sentencias `UPDATE` ejecutadas desde la aplicación o desde la consola.

Relaciones entre las tablas

Las tablas se conectan por medio de 31 llaves foráneas. Estas llaves garantizan que no se pueda registrar, por ejemplo, una cotización para una solicitud que no existe ni una orden de un comprador inexistente. La Tabla 7 las lista.

Tabla 7

Llaves foráneas definidas en la base de datos

Tabla hija	Columna	Tabla referenciada
usuarios	id_rol	roles
solicitudes_compra	id_solicitante	usuarios
solicitudes_compra	id_responsable	usuarios
solicitudes_compra	id_estado_solicitud	estados_solicitud
solicitudes_compra	id_prioridad	prioridades
detalle_solicitud	id_solicitud	solicitudes_compra
detalle_solicitud	id_tipo_item	tipos_item
aprobaciones	id_solicitud	solicitudes_compra
aprobaciones	id_usuario	usuarios
aprobaciones	id_tipo_decision	tipos_decision
cotizaciones	id_solicitud	solicitudes_compra
cotizaciones	id_proveedor	proveedores
cotizaciones	id_estado_cotizacion	estados_cotizacion
detalle_cotizacion	id_cotizacion	cotizaciones
detalle_cotizacion	id_detalle_solicitud	detalle_solicitud
ordenes_compra	id_solicitud	solicitudes_compra
ordenes_compra	id_cotizacion	cotizaciones
ordenes_compra	id_comprador	usuarios
ordenes_compra	id_estado_orden	estados_orden
detalle_orden_compra	id_orden	ordenes_compra
detalle_orden_compra	id_detalle_solicitud	detalle_solicitud
detalle_orden_compra	id_detalle_cotizacion	detalle_cotizacion
recepciones	id_orden	ordenes_compra
recepciones	id_usuario	usuarios

Tabla hija	Columna	Tabla referenciada
recepciones	id_estado_recepcion	estados_recepcion
detalle_recepcion	id_recepcion	recepciones
detalle_recepcion	id_detalle_orden	detalle_orden_compra
facturas	id_proveedor	proveedores
facturas	id_orden	ordenes_compra
facturas	id_estado_factura	estados_factura
auditoria	id_usuario	usuarios

Nota. Elaboración propia a partir del script final.

Reglas de integridad y cálculos automáticos

La base de datos controla la calidad de la información con varios mecanismos. Las llaves primarias identifican cada registro de forma única. Las restricciones `UNIQUE` impiden duplicar documentos, correos, códigos de solicitud, de cotización y de orden, identificaciones fiscales y nombres de catálogo. Las restricciones `NOT NULL` obligan a llenar los datos esenciales y los valores `DEFAULT` completan solos las fechas y los estados activos.

Las restricciones `CHECK` validan que las cantidades sean positivas, que los valores monetarios no sean negativos, que la fecha de vencimiento de una cotización no sea anterior a su fecha de emisión, que la cantidad aceptada no supere la recibida y que la acción de auditoría sea solo `INSERT`, `UPDATE` o `DELETE`.

Por último, las columnas generadas calculan los valores derivados: `valor_total` en cotizaciones, órdenes y facturas; `subtotal` en los detalles de cotización y de orden; y `cantidad_rechazada` en la recepción. Como las calcula el propio gestor, se evitan los errores de suma que ocurrirían si cada usuario digitara los totales. Por ejemplo, si una cotización tiene subtotal 1.000.000, descuento 50.000 e impuestos 190.000, MariaDB guarda un `valor_total` de 1.140.000.

Ejemplo de consulta

Como ejemplo de uso, la siguiente consulta muestra cada solicitud de compra con el nombre de quien la solicitó, su estado y su prioridad. Para obtener esos datos se unen cuatro tablas mediante `JOIN`, aprovechando las llaves foráneas:

```
SELECT s.codigo_solicitud,
       CONCAT(u.nombres, ' ', u.apellidos) AS solicitante,
       e.nombre_estado AS estado,
       p.nombre_prioridad AS prioridad,
       s.fecha_solicitud
FROM solicitudes_compra s
JOIN usuarios u ON u.id_usuario = s.id_solicitante
JOIN estados_solicitud e
     ON e.id_estado_solicitud = s.id_estado_solicitud
JOIN prioridades p ON p.id_prioridad = s.id_prioridad
ORDER BY s.fecha_solicitud DESC;
```

Verificación de la estructura final

Para comprobar el resultado se ejecutó `SHOW TABLES;`, que listó las 22 tablas esperadas, y se aplicó `DESCRIBE` a cada una para revisar sus columnas, llaves y valores por defecto. En las columnas generadas, la columna `Extra` muestra `STORED GENERATED`, y en las llaves foráneas la columna `Key` muestra `MUL`. También se usó `SHOW CREATE TABLE usuarios;`, que confirmó la llave primaria, los índices únicos de `documento` y `correo`, la llave foránea `fk_usuarios_roles` y que la tabla quedó con motor InnoDB y la intercalación `utf8mb4_unicode_ci`.

```
SHOW TABLES;
DESCRIBE usuarios;
DESCRIBE cotizaciones;
SHOW CREATE TABLE usuarios;
```